Code You Never Write -Final Report

Name: Syeda Malika Zainab Kazmi **AI Tool(s) Used:** Claude (Anthropic)

Project 1: Money Detective

Problem it solves: Instead of tracking spending going forward, this project hunts for money leaks hidden in past transaction history — recurring charges, duplicate payments, and forgotten subscriptions that generic budgeting apps can't detect because the rules are personal.

AI Tool Used: Claude

Initial Prompt:

"Here is my transaction history (date, description, amount). Write a Python script that finds: (1) recurring/subscription charges, (2) duplicate or repeated payments on the same day, (3) possible forgotten subscriptions (charged once, never seen again). Output a clear summary report."

No further prompt iteration was needed — the first version worked correctly.

Verification: Checked the script's monthly totals against known baseline figures by hand:

- May total: known 32,850 vs script 32,850
- June total: known 39,740 vs script 39,740
- A deliberately known duplicate charge (Netflix, 21 May, charged twice) was correctly caught by the script.

What worked / didn't: The script correctly identified all recurring charges and the one duplicate. No forgotten subscriptions were found in this dataset — a legitimate result, not a failure.

Final result: Found one duplicate charge (PKR 1,500 wasted) and a clear map of 5 recurring monthly charges. Action: review the duplicate with the payment provider.

Screenshot:

```
============================================================
MONEY DETECTIVE REPORT
============================================================

Total transactions: 35
Total spend: 72,590

--- Monthly Totals (verify these against your own records) ---
  2026-05: 32,850
  2026-06: 39,740

--- Possible Duplicate Charges (2 rows) ---
      Date          Description  Amount
2026-05-21 Netflix Subscription    1500
2026-05-21 Netflix Subscription    1500
  Estimated wasted amount from duplicates: 1,500

--- Recurring Charges / Subscriptions (5 found) ---
                         Description  Amount  Occurrences
                 Netflix Subscription    1500           4
                Mobile Balance - Jazz    1000           3
Doctor Appointment - Dr. Ahmed Clinic    2500           2
                     Electricity Bill    6800           2
                      Spotify Premium     850           2

--- Possible Forgotten Subscriptions (0 found) ---
  None found.

============================================================
End of report.
============================================================
```

Project 2: What's My Grade, Really

Problem it solves: Generic grade apps don't know a specific teacher's actual rules — category weights, dropped lowest scores. This project encodes those exact rules to find the true current grade and the score needed on the Final to reach a target.

AI Tool Used: Claude

Initial Prompt:

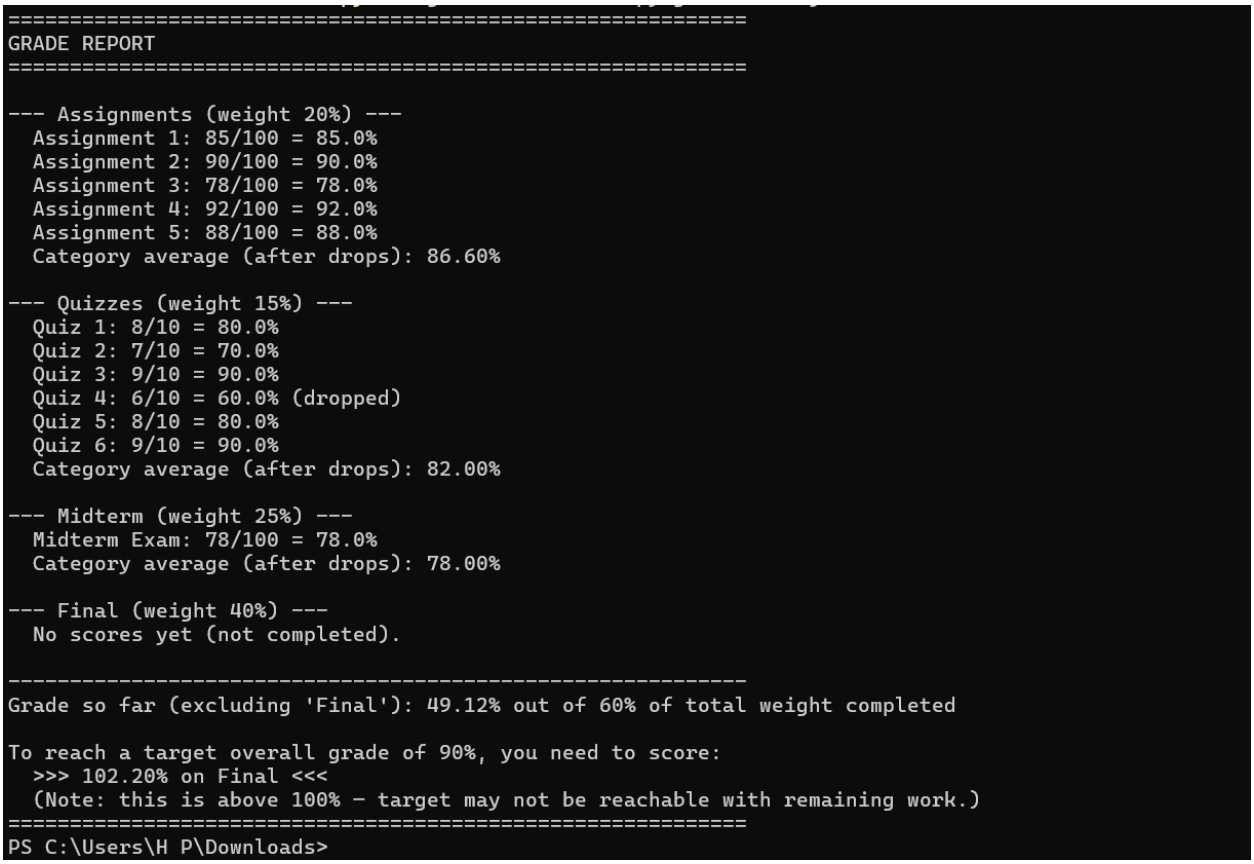
"Here are my scores (assignments, quizzes, midterm) and my teacher's grading policy (weights per category, dropped lowest quiz score). Write a script that calculates my current grade, and tell me what score I need on the final exam to reach a 90% target."

Verification: Manually recalculated the Quizzes category by hand: scores 8,7,9,6,8,9 out of 10 → percentages 80,70,90,60,80,90 → lowest (60%) dropped per policy → average of remaining 5 = 82.00%. Script output: 82.00% — exact match.

What worked / didn't: The drop-lowest-quiz logic and the reverse calculation (score needed on remaining category) both worked correctly. The initial 90% target turned out to require 102.20% on the Final — mathematically impossible — an honest finding rather than a script error.

Final result: A target grade of 90% is not currently reachable; a target in the 75–85% range is realistic given current standing. Exact scores needed for several target grades were calculated (e.g. 77.20% on the Final for an 80% overall grade).

Screenshot:



Project 3: The Books Don't Match

Problem it solves: Reconciling a hand-counted, known-correct total (a class farewell party fund) against a messy digital payment record with inconsistent nicknames, using personal rules to interpret ambiguous entries.

AI Tool Used: Claude

Initial Prompt:

"Here is my expected total (hand-counted) and a messy list of digital payment records with inconsistent names. Here are my personal rules for matching ambiguous names to real people.

Write a script that reconciles the two, finds the gap, and identifies which people still need to pay or are short."

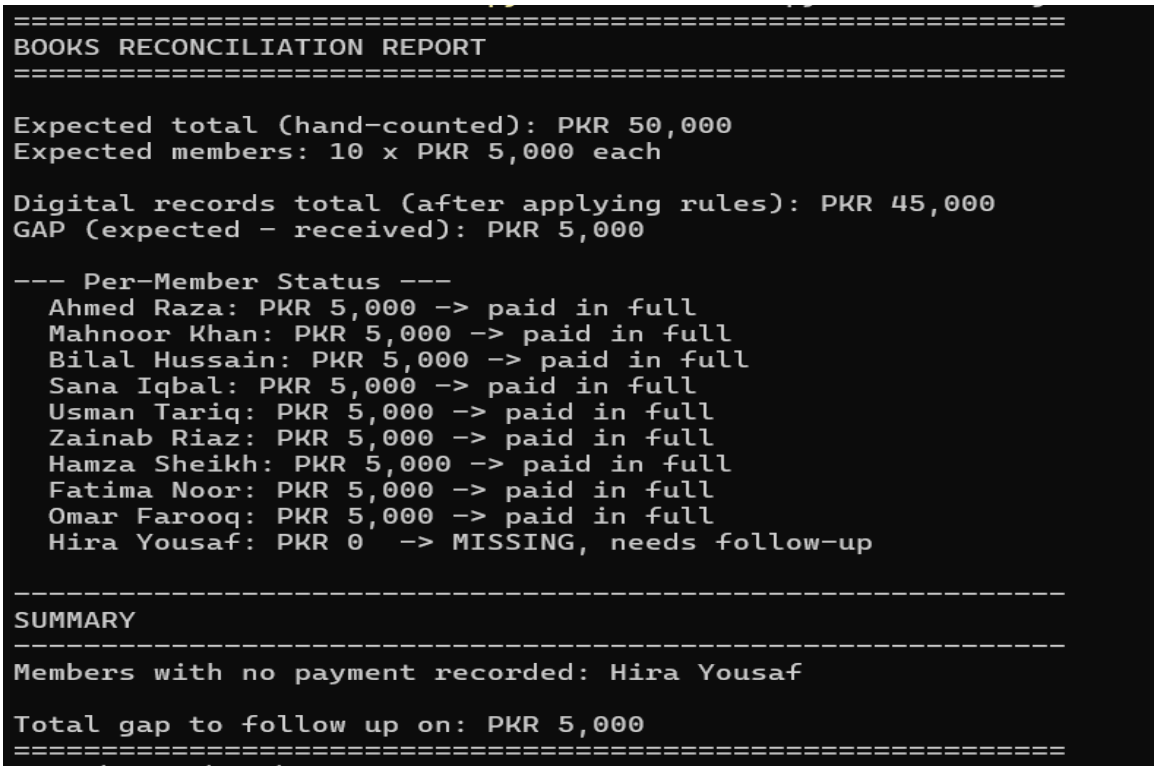
Improved prompt / fix: During verification, a bug was found: the script failed to match an ambiguous "unknown transfer" entry to the correct member (Sana Iqbal) due to an exact-string-match issue, which incorrectly flagged her as underpaid. Asked the AI to fix the matching logic to use substring matching instead of exact equality — this corrected the issue.

Verification: The overall gap (PKR 5,000) matched the known missing member (Hira Yousaf) exactly, both before and after the fix. After the fix, Sana Iqbal was correctly shown as paid in full — matching what was already known about her payment.

What worked / didn't: Name-alias resolution worked correctly on the first try. The special-case rule matching had a real bug that was only caught by checking the per-member breakdown against known facts, even though the headline total already looked correct — a direct demonstration of the assignment's core lesson.

Final result: Gap of PKR 5,000 identified; Hira Yousaf confirmed as the person needing follow-up; all other 9 members confirmed paid in full.

Screenshot:



Project 4: Organize the Mess (The Files You Forgot)

Problem it solves: A real, 18,550-file Downloads folder accumulated over roughly a year had become cluttered with duplicate downloads and forgotten installers. This project required real file-operation safety discipline, since a careless script could delete or misplace real files.

AI Tool Used: Claude

Initial Prompt:

"Here is my messy Downloads folder. Write a script that finds duplicate files (by content, not just filename), flags very large files (>100MB), and groups files by type. Show me a full dry-run plan first — every operation you would perform — and don't touch anything until I approve it. Never delete or overwrite my original files."

Improved prompts / fixes (iterative, based on real errors):

1. First execution attempt ran out of disk space trying to physically copy every file, including multi-gigabyte installers. Asked the AI to fix this — files over 200MB are now only recorded, not copied.
2. A UnicodeEncodeError occurred on filenames with special characters. Asked the AI to fix file encoding to UTF-8 across all report writes.
3. After the disk filled up completely on a second attempt, the AI redesigned the execution step to be report-only by default, removing the disk-space risk entirely for very large folders.

Verification (safety steps followed):

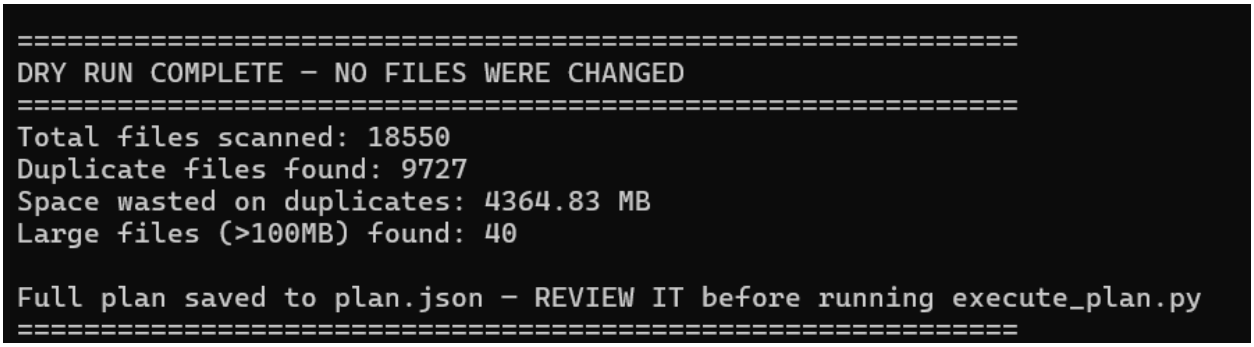
1. Copied the entire Downloads folder before running any script.
2. Ran a dry-run scan that only reads files and produces a plan — nothing was touched.
3. Reviewed the plan (plan.json) before execution.
4. Approved execution explicitly (typed "YES").
5. Verified afterward that the original folder was completely unchanged, and the new Organized folder was structured correctly.

What worked / didn't: Content-hash duplicate detection correctly found genuine duplicates, including a video file saved under three different names (~750MB wasted). Two real bugs (disk space, encoding) were caught only by actually running the script against real data, and both were fixed through the same "client describes the problem, AI fixes it" loop the assignment is built around.

Final result: 18,550 files scanned; 9,727 duplicate files found (~4.3 GB of wasted space); 40 large files (>100MB) identified. Files were organized into 9 type-based folders as copies, with

the original folder fully intact. Action: review duplicates_report.txt to reclaim ~4.3 GB by deleting confirmed duplicates.

Screenshot:



Overall Reflection

Across all four projects, the recurring lesson was the same: **the AI's first answer was often plausible-looking but not automatically correct.** In Project 3, a matching bug produced a wrong per-person result even though the total looked right. In Project 4, the first two approaches failed against real-world constraints (disk space, character encoding) that only became visible when actually run against real data. In every case, the fix came from checking the output against something already known to be true — not from trusting the AI's confidence in its own answer. That verification habit, more than any single script, was the actual skill this assignment was teaching.